

Experiment No. 9

Aim: Extend the ML CI workflow by packaging a trained model behind a simple API, creating a deployable artifact/container and automatically verifying the deployed inference service.

Tools:

- Python 3 and Docker.
- Jenkins.
- Git/GitHub.
- FastAPI, Uvicorn, scikit-learn and joblib.
- Model artifact generated by Experiment 8 or an equivalent saved model.

Theory:

A trained ML model saved as an artifact (e.g., model.pkl) sitting on a hard drive has no real-world use on its own — it needs to be made accessible so other applications can send it data and receive predictions. This process is called Deployment.

Deployment wraps the trained model inside a web server (API) — here built using Flask — which exposes an endpoint (e.g., /predict) that accepts input data over HTTP and returns the model's prediction as a response. This API is then packaged into a Docker container, making it portable and able to run consistently on any machine, and exposed on a specific port (5000) so it can serve requests 24/7.

Jenkins automates this entire flow: it pulls the latest code from GitHub, retrains the model to ensure freshness, builds a new Docker image containing the Flask API and the model, removes any old running container to avoid port conflicts, and launches the new container in the background. This makes Jenkins the "middleman" that continuously pushes updated models from development straight into a live production server, without manual intervention — a core principle of MLOps deployment automation.

Unlike Experiment 8, where the pipeline stopped at producing and testing model.pkl, Experiment 9 extends the pipeline one step further: from a static artifact to a live, queryable inference service.

Steps Performed

1. Updated requirements.txt to include Flask alongside scikit-learn and joblib.
2. Created app.py — a Flask web API that loads model.pkl and exposes a /predict endpoint to accept input features and return predictions in JSON format.
3. Updated Dockerfile to expose port 5000 and run app.py (the live server) instead of train.py.
4. Pushed the updated files to GitHub:
git add .

- ```
git commit -m "Add Flask API for model deployment"
```
- ```
git push origin main
```
- Created a New Freestyle Project in Jenkins:
New Item → named ML-Deployment-Exp9 → selected Freestyle project → OK
 - Linked the same GitHub repository under Source Code Management, with Branches to Build set to */main.
 - Added a Build Step (Execute Windows batch command) containing the full deployment sequence:

```
pip install -r requirements.txt
```

```
python train.py
```

```
docker build -t ml-live-api:v1 .
```

```
docker rm -f ml_production_server
```

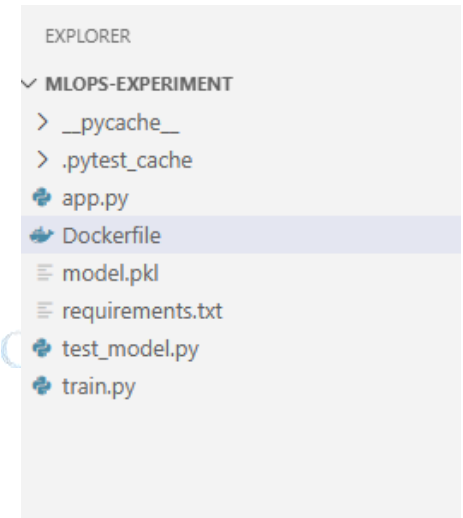
```
docker run -d -p 5000:5000 --name ml_production_server ml-live-api:v1
```
 - Saved and clicked Build Now. Jenkins retrained the model, built a fresh Docker image, removed any old container, and launched a new container serving the Flask API on port 5000.
 - Tested the live API using PowerShell:
 Invoke-RestMethod -Uri <http://localhost:5000/predict> -Method Post -ContentType "application/json" -Body '{"features": [5.1, 3.5, 1.4, 0.2]}'
 Received a response such as {"prediction": 0}, confirming the containerized model was successfully deployed and actively serving predictions.

Code:

Train.py:

```
import joblib
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

def train():
    # Load dataset
    iris = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(
        iris.data, iris.target, random_state=42
    )
    # Train lightweight model
    clf = RandomForestClassifier(n_estimators=10, random_state=42)
    clf.fit(X_train, y_train)
    # Evaluate basic accuracy
    accuracy = clf.score(X_test, y_test)
    print(f"Model Accuracy: {accuracy * 100:.2f}%")
```



```
# Save the model artifact
joblib.dump(clf, "model.pkl")
print("Model successfully saved as model.pkl")

if __name__ == "__main__":
    train()

test_model.py:
import os
import joblib

def test_model_artifact_exists():
    assert os.path.exists("model.pkl"), "Error: model.pkl artifact was not
created."

def test_model_inference():
    clf = joblib.load("model.pkl")
    sample_input = [[5.1, 3.5, 1.4, 0.2]]
    prediction = clf.predict(sample_input)
    assert len(prediction) == 1, "Error: Model failed to generate a prediction."
```

app.py:

```
from flask import Flask, request, jsonify
import joblib
import os

app = Flask(__name__)

# Load the trained model artifact
MODEL_PATH = "model.pkl"
if os.path.exists(MODEL_PATH):
    model = joblib.load(MODEL_PATH)
else:
    model = None

@app.route('/', methods=['GET'])
def home():
    return "ML Model Deployment API is Live! Send POST requests to /predict"

@app.route('/predict', methods=['POST'])
def predict():
    if not model:
```

```

    return jsonify({"error": "Model not found. Train the model first."}), 500
try:
    data = request.get_json()
    features = data['features']
    prediction = model.predict([features])
    return jsonify({"prediction": int(prediction[0])})
except Exception as e:
    return jsonify({"error": str(e)}), 400

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

Dockerfile:

```

FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]

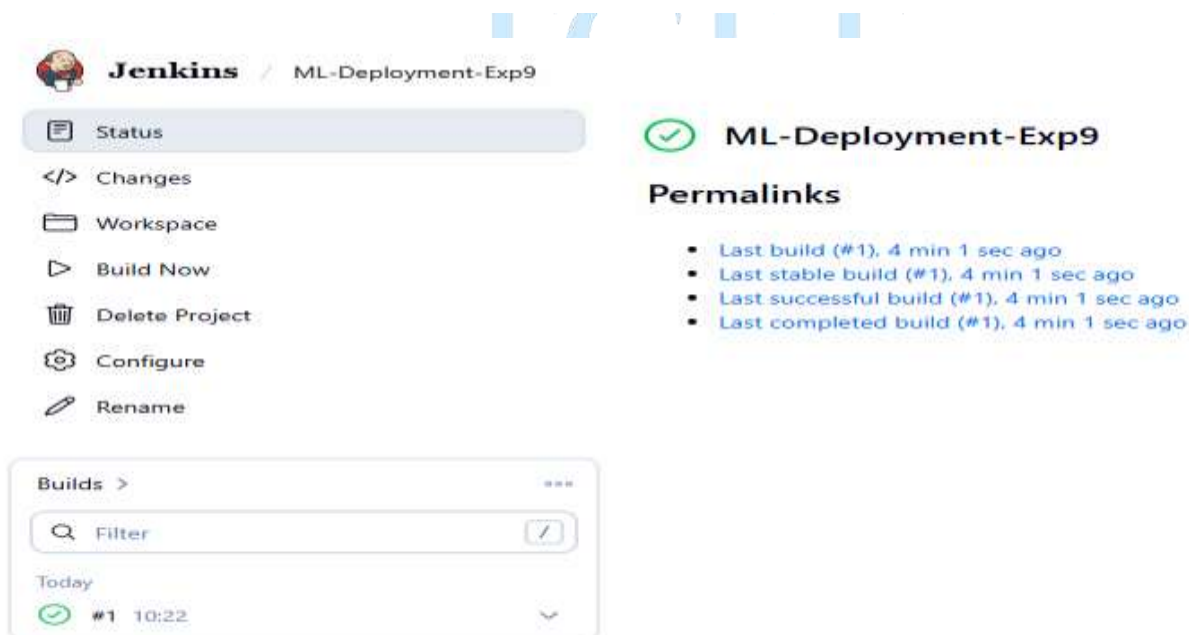
```

Output:

```

PS C:\Users\Lab 310\Desktop\mlops-experiment> Invoke-WebRequest -Uri http://localhost:5000/predict -Method Post -ContentType "application/json" -Body '{"features": [5.1, 3.5, 1.4, 0.2]}'
prediction
-----
0

```



The image shows the Jenkins web interface for a project named "ML-Deployment-Exp9". The interface includes a sidebar with navigation options: Status, Changes, Workspace, Build Now, Delete Project, Configure, and Rename. The main area displays the project name with a green checkmark icon, indicating a successful build. Below this, there are "Permalinks" for the last build, last stable build, last successful build, and last completed build, all showing a time of 4 min 1 sec ago. At the bottom, there is a "Builds" section with a search filter and a list of builds, showing a single build (#1) completed today at 10:22.

```
Jenkins ML-Deployment-Exp0 #1
```

```
#E 16.17
#E 16.17 [notice] A new release of pip is available: 23.0.1 -> 26.0.1
#E 16.17 [notice] To update, run: pip install --upgrade pip
#E DONE 16.0s

#E [5/5] COPY --
#E DONE 0.1s

#E exporting to image
#E exporting layers
#E exporting layers 1.0s done
#E writing image sha256:f06ecc1364b1d708b8e1b21802ac5d6c8d7db047f9e014de7556714af6a2f done
#E naming to docker.io/library/ml-live-api:v1 0.0s done
#E DONE 1.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/c8q9ctsts1priz701uqfzg

C:\Users\Lab 118\Jenkins\workspace\ML-Deployment-Exp0\docker rm -f ml_production_server
error response from daemon: no such container: ml_production_server

C:\Users\Lab 118\Jenkins\workspace\ML-Deployment-Exp0\docker run -d -p 5000:5000 --name ml_production_server ml-live-api:v1
7444766dd7b4e8bd1300d024597451e0767890088b6cf1e957f3e29183c

C:\Users\Lab 118\Jenkins\workspace\ML-Deployment-Exp0\exit 0
Finished: SUCCESS
```

Images / ml-live-api:v1

ml-live-api:v1 IN USE
f06ecc1364b1

Layers (16)

		Size
0	# debian.sh --arch 'amd64' out/ 'trixie' '@1760918400'	78.62 MB
1	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr_...	0 B
2	ENV LANG=C.UTF-8	0 B
3	RUN /bin/sh -c set -eux; apt-get update; apt-get install -y --no...	3.81 MB
4	ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3102...	0 B
5	ENV PYTHON_VERSION=3.9.25	0 B
6	ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8402a...	0 B


docker desktop
PERSONAL

CBCGS
🔔
🌐
⚙️
📦
Sign in

Containers

Images

Volumes

Builds

Docker Hub

Docker Scout

Extensions

Containers / determined_gates


determined_gates

eaa4c435b8f8
ml-live-api:v1

STATUS

Running (1 second ago)





Logs

Inspect

Bind mounts

Exec

Files

Stats

```

2025-09-08 18:27:33 /usr/local/lib/python3.9/site-packages/sklearn/base.py:389: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.0.1, when using version 1.6.1. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
2025-09-08 18:27:33 warnings.warn()
2025-09-08 18:27:33 /usr/local/lib/python3.9/site-packages/sklearn/base.py:389: InconsistentVersionWarning: Trying to unpickle estimator RandomForestClassifier from version 1.0.1, when using version 1.6.1. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
2025-09-08 18:27:33 warnings.warn()
2025-09-08 18:27:33 * Serving Flask app 'app'
2025-09-08 18:27:33 * Debug mode: off
2025-09-08 18:27:33 WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
2025-09-08 18:27:33 * Running on all addresses (0.0.0.0)
2025-09-08 18:27:33 * Running on http://127.0.0.1:5000
2025-09-08 18:27:33 * Running on http://172.17.0.3:5000
2025-09-08 18:27:33 Press CTRL+C to quit

```

Learning Objective:

- Understand CD for an ML inference service.
- Package a model with an API.
- Containerize an ML inference service.
- Perform post-deployment smoke testing.

Conclusion:

For Faculty Use:

Correction Parameters	Formative Assessment [40%]	Timely Completion of Practical [40%]	Attendance/ Learning Attitude [20%]	